

RAPPORT DE PROJET POO

THEME :

SIMNET : Simulateur de réseau en entreprise

Membres du groupe 1 :

- ♦ BITOUKOUA OUSMANE Lame (**Chef**)
- ♦ ONGUENE ALIMA Anne Leslie
- ♦ MACHEU TALLA Sandra
- ♦ TSAFAC TEUFAC Belvira Lins

Etudiants d'ingé 3 SRT

Sous la supervision de : M FEDHIM Stéphane

2025-2026

Table des matières

INTRODUCTION	3
1. Présentation du groupe et des rôles	4
2. Diagramme de classes	4
<i>Relations entre les classes</i>	5
3. Justification des choix de conception	6
4. Problèmes et solutions	7
CONCLUSION	8

INTRODUCTION

Etant donné le fait que l'évolution constante des infrastructures numériques, une bonne compréhension des architectures réseaux et de leur sécurisation est un enjeu majeur pour les étudiants ingénieurs en systèmes, réseaux et télécommunications que nous sommes. C'est dans cette optique et dans le cadre des projets de fin de semestre de l'unité d'enseignement Programmation Orientée Objet (**POO**) que nous avons reçu comme tâche de réaliser le projet SIMNet : un simulateur de réseau d'entreprise entièrement orienté objet en langage Python. Il s'agit pour nous étudiants ingénieurs de niveau 3 de l'occasion parfaite pour pouvoir implémenter nos connaissances en POO, un travail challengeant mais qui pourra se résoudre grâce au travail d'équipe et au bon sens. Ce rapport présente globalement les travaux de notre groupe, en mettant en exergue la modélisation des classes, le choix des conceptions architecturales et bien sûr les difficultés techniques rencontrées.

1. Présentation du groupe et des rôles

Pour mener à bien notre projet, nous avons réparti le travail de façon à ce que chaque membre de l'équipe réalise une partie bien précise du projet. Les membres du groupe et leurs différents rôles sont :

Noms & Prénoms	Rôle	Responsabilités
BITOUKOUA Ousmane Lamé	Chef de groupe Responsable de la structure des paquets et topologies	Conception et développement de topologie.py et paquets.py . Implémentation de la classe Topologie (gestion des équipements et des liens), de la classe Lien, de la classe Paquet avec gestion des services réseau, et de l'algorithme de routage BFS. Coordination générale du groupe
MACHEU TALLA Sandra	Modélisation de la classe mère Equipement et gestion des attributs physiques	Conception et développement de equipements.py . Définition de la hiérarchie de classes : classe abstraite Equipement et des classes filles. Mise en place de l'abstraction, encapsulation et du polymorphisme
ONGUENE ALIMA Anne Leslie	Implémentation du module de monitoring et d'affichage des flux réseaux.	Conception et développement de moniteur.py et main.py . Implémentation de la classe Moniteur, du menu interactif et de l'intégration finale de tous les modules
TSAFAC TEUFAC Belvira Lins	Gestion de la journalisation d'audit (logs) et sécurisation des accès (Authentification).	Conception et implémentation du module Pare-feu (securite.py). Implémentation des classes RegleFiltrage, JournalFirewall, Authentification et Firewall (sécurité). Gestion du filtrage par protocole, IP source et port, avec journalisation horodatée de toutes les décisions

Le développement s'est organisé en phases parallèles : chaque membre a travaillé sur son module de manière indépendante, en respectant une convention de nommage commune définie dès le premier jour (snake_case pour les méthodes, PascalCase pour les classes, docstrings obligatoires) via le diagramme de classes, ce qui a facilité une gestion prudente des importations notamment entre securite.py et equipements.py, résolue par un bloc try/except. Le versioning a été assuré via Git, avec des commits réguliers sur la branche group_1 du dépôt forké depuis le dépôt officiel de l'enseignant.

2. Diagramme de classes

Le diagramme de classes de SIMNet s'articule autour d'une hiérarchie centrée sur la classe abstraite Equipement, complétée par des classes de service assurant la simulation,

sécurité et la supervision du réseau. Pour des soucis d’affichage et de lecture, le diagramme UML réalisé à l’aide de draw.io sera fourni en annexe.

Classe	Fichier	Type	Rôle
Equipement	Equipements .py	Abstraite	Classe mère de tous les équipements réseau
Routeur	Equipements .py	Concrète	Gère table de routage et interfaces
Switch	Equipements .py	Concrète	Segmentation réseau via VLANs
Serveur	Equipements .py	Concrète	Expose des services réseau (HTTP, SSH...)
Firewall	Securite.py	Concrète	Concrète Filtre le trafic, authentification, journal
Point_Acces_wifi	Equipements .py	Concrète	Gestion connexions sans fil (SSID, fréquence
Terminal	Equipements .py	Concrète	Client final (PC, smartphone...)
Paquet	Paquets.py	Concrète	Unité de données avec service, protocole, priorité
Lien	Topologie.py	Concrète	Connexion physique avec bande passante et latence
Topologie	topologie.py	Concrète	Carte du réseau + algorithme BFS + envoi de paquets
RegleFiltrage	securite.py	Concrète	Règle unitaire : IP source, protocole, port, action
JournalFiltrage	securite.py	Concrète	Historique horodaté des décisions du firewall
Authentification	securite.py	Concrète	Contrôle d'accès à la configuration du firewall
Moniteur	Moniteur.py	Concrète	Historique des 10 derniers paquets, rapport

Relations entre les classes

Héritage (relation « EST UN »)

Les six classes concrètes d'équipements héritent toutes de la classe abstraite Equipement. Elles récupèrent automatiquement les attributs communs (nom, adresse_ip, marque, statut) et les méthodes de base (activer (), desactiver (), __str__ ()), tout en implémentant obligatoirement la méthode abstraite afficher_infos () selon leurs spécificités. Le Firewall de securite.py hérite également d'Equipement, ce qui lui confère son identité d'équipement réseau à part entière.

Composition (relation « CONTIENT »)

La classe Topologie est en composition avec Equipement et Lien : elle contient et gère les listes d'équipements et de liens, qui n'ont pas d'existence autonome en dehors du réseau. De même, le Firewall (securite.py) est en composition avec JournalFirewall et Authentification — ces

deux modules sont instanciés directement dans le constructeur du Firewall et lui appartiennent. Association (relation « UTILISE ») La classe Topologie utilise la classe Paquet dans sa méthode `envoyer_paquet ()` pour simuler le transit. La classe MoniteurReseau reçoit des paquets et des résultats qu'elle enregistre, sans en être propriétaire. Le MenuConsole (`main.py`) orchestre toutes les classes sans les posséder — il instancie Topologie, Moniteur et Firewall, et délègue chaque action à la classe compétente.

3. Justification des choix de conception

Cette partie du travail met en lumière les raisons qui nous ont poussé à choisir les mécanismes présents dans notre code.

Structure	Justification
Abstraction	La classe Equipement est définie comme une classe abstraite via le module <code>abc</code> de la bibliothèque standard Python. Elle déclare la méthode abstraite <code>afficher_infos ()</code> avec <code>@abstractmethod</code> , ce qui impose à toute sous-classe concrète de l'implémenter. Il est ainsi impossible d'instancier un "équipement générique" — seuls les équipements réels (Routeur, Switch, etc.) peuvent être créés
Encapsulation	Tous les attributs des classes sont déclarés privés avec le préfixe underscore (<code>self._nom</code> , <code>self._adresse_ip</code> , <code>self._statut...</code>). L'accès contrôlé est assuré par des propriétés Python (<code>@property</code>) exposant les valeurs en lecture, sans permettre de modification directe depuis l'extérieur
Héritage	L'héritage est mis en œuvre à deux niveaux dans SIMNet. Premièrement, chaque classe d'équipement hérite d'Equipement via la syntaxe <code>class Routeur (Equipement)</code> . Le constructeur de la classe mère est systématiquement appelé avec <code>super ().__init__ ()</code> , évitant toute duplication des attributs communs
Polymorphisme	Le polymorphisme se manifeste de plusieurs façons dans SIMNet. La méthode <code>afficher_infos ()</code> porte le même nom dans toutes les classes, mais produit un affichage différent selon l'équipement réel : un Routeur affiche sa table de routage, un Switch ses VLANs, un Serveur ses services exposés, etc. La méthode <code>envoyer_paquet ()</code> de Topologie exploite également le polymorphisme : elle parcourt le chemin équipement par équipement et vérifie dynamiquement avec <code>hasattr (équipement, 'filtrer')</code> si l'équipement courant est un

	Firewall, sans avoir besoin de connaître son type exact à l'avance
Enumérations	Les protocoles réseau (TCP, UDP, ICMP) et les actions du firewall (AUTORISER, BLOQUER) sont modélisés avec la classe Enum de la bibliothèque standard. Ce choix évite les erreurs de frappe sur les chaînes de caractères et rend le code plus robuste et plus lisible. La classe Paquet gère intelligemment les deux formes (Enum ou chaîne) grâce à <code>isinstance()</code> , assurant la compatibilité entre modules

L'algorithme utilisé pour le routage de paquets est l'algorithme de parcours en largeur **BFS (Breadth-First Search)** :

Il est implémenté dans la méthode `trouver_chemin()` de la classe `topologie` en utilisant la structure `deque` de la bibliothèque standard `collections`. Ce choix se justifie par plusieurs raisons :

- **Optimalité en nombre de sauts** : le BFS garantit de trouver le chemin avec le moins d'équipements intermédiaires, ce qui est la métrique la plus naturelle dans un réseau local d'entreprise.
- **Simplicité d'implémentation** : le BFS ne nécessite aucune bibliothèque externe et s'exprime clairement avec une file (`deque`) et un ensemble de visités.
- **Adéquation au contexte** : pour un simulateur pédagogique, un algorithme lisible et justifiable est préférable à un Dijkstra dont la complexité n'apporte pas de valeur ajoutée à l'échelle d'un LAN.

4. Problèmes et solutions

Durant la réalisation de ce projet, nous avons fait face à quatre problèmes majeurs

- **La compatibilité Enum / str**
Le protocole d'un paquet pouvait être un Enum (Protocole TCP) ou une chaîne (TCP) selon le module d'origine. Pour résoudre ce problème nous avons procédé à une vérification systématique avec **`isinstance()` (protocole Enum)** dans les méthodes de comparaisons du Firewall et du paquet
- **La recherche par IP dans topologie**
La méthode **`trouver_equipement_par_ip()`** accédait à **`equipement.Ip`** au lieu de **`equipement.adresse_ip`**, causant des **`AttributeError`**. Pour y remédier, nous avons uniformiser l'accès via les propriétés (**`@property`**) définies dans `Equipement`, assurant un nom cohérent dans tout le projet.

- **L'intégration des modules**

Nous avons développé en parallèle avec des conventions légèrement différentes (noms de paramètres, ordres des arguments). Nous avons donc ainsi organisé une session d'intégration commune, de revue du code et de standardisation des interfaces entre classes

CONCLUSION

Le développement de SIMNet nous a permis de mettre en pratique l'ensemble des concepts fondamentaux de la programmation orientée objet c'est-à-dire abstraction via une classe mère abstraite, encapsulation des attributs sensibles, héritage en cascade avec la fonction super, polymorphisme pour le filtrage et l'affichage, et composition pour la modélisation du réseau.

Au-delà de la technique, ce projet nous a également appris à travailler en équipe sur une base de code commune à travers Git, la définition des conventions partagées et la gestion des incompatibilités entre modules développés en parallèle. Le simulateur ; obtenu bien que pédagogique, reflète fidèlement les mécanismes d'une infrastructure réseau réelle à savoir, topologie, routage, sécurité et supervision :